

FUNDAMENTOS DE PRORGRAMAÇÃO

Assincronismo



Teach
3035

Índice

Assincronismo.....	03
<u>Callbacks</u>	03
<u>Promises</u>	04
<u>Async/Await</u>	06

Assincronismo



O **assincronismo** em JavaScript permite que o código execute tarefas sem bloquear a execução subsequente. Em vez de esperar que uma tarefa seja concluída antes de prosseguir, o código assíncrono inicia a tarefa e continua executando o restante do programa. Quando a tarefa é concluída, uma função de retorno é chamada para lidar com os resultados. Isso evita que o programa fique bloqueado e resulta em uma melhor experiência para o usuário.

Callbacks



Um callback é uma função que é passada como argumento para outra função e é executada posteriormente, geralmente quando uma operação assíncrona é concluída. Veja um exemplo simples de uso de callbacks em JavaScript:

```
function asyncRequest (callback) {  
  setTimeout(function() {  
    const result = "Dados da requisição";  
    callback(result);  
  }, 2000);  
}  
  
function callbackFunction(data) {  
  console.log(data);  
}  
  
asyncRequest(callbackFunction);
```

Callbacks



A função `asyncRequest` simula uma requisição assíncrona com um atraso de 2 segundos. Ela recebe um callback como argumento, que será chamado quando a requisição for concluída, passando os dados da requisição como parâmetro. A função `callbackFunction` é definida para imprimir os dados recebidos no `console`. Por fim, a função `asyncRequest` é chamada, passando `callbackFunction` como argumento, para executar a requisição assíncrona e imprimir os dados resultantes no console.

Promises



São objetos que representam o resultado futuro de uma operação assíncrona. Elas foram introduzidas para lidar de forma mais eficiente com o assincronismo e simplificar o código em comparação aos callbacks.

O Sintaxe

```
const myPromise = new Promise(function(resolve, reject) {  
  // Lógica assíncrona  
});
```

Promises



○ Exemplo

```
function asyncRequest() {
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      const success = true;

      if (success) {
        resolve("Dados da requisição");
      } else {
        reject("Erro na requisição");
      }
    }, 2000);
  });
}

asyncRequest()
  .then(function(result) {
    console.log(result); // Imprimindo os dados da requisição bem-sucedida
  })
  .catch(function(error) {
    console.error(error); // Imprimindo a mensagem de erro em caso de falha
    na requisição
  });
```

A função `asyncRequest` é um exemplo de função assíncrona que retorna uma `Promise`, que simula uma requisição assíncrona com um atraso de 2 segundos. Se a requisição for bem-sucedida, a `Promise` é resolvida com os dados da requisição. Caso contrário, se ocorrer um erro na requisição, a `Promise` é rejeitada com a mensagem de erro. O código subsequente utiliza os métodos `then` e `catch` para tratar o resultado da `Promise`, imprimindo os dados da requisição em caso de sucesso e a mensagem de erro em caso de falha.

Async/Await



O `async` é usado para declarar uma função assíncrona. Uma função assíncrona retorna sempre uma `Promise`. Dentro dessa função, podemos usar a palavra-chave `await` para pausar a execução e aguardar a resolução de uma `Promise`. Isso permite que o código pareça síncrono, mesmo executando operações assíncronas.

○ Sintaxe

```
async function functionName() {  
  try {  
    // Código assíncrono usando await  
    const resultado = await asyncRequest();  
    // Restante do código que depende do resultado  
  } catch (error) {  
    // Tratamento de erros  
  }  
}
```

○ Exemplo

```
async function getData() {  
  try {  
    const response = await  
    fetch("https://jsonplaceholder.typicode.com/todos/1");  
    if (!response.ok) {  
      throw new Error("Erro ao obter os dados");  
    }  
  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}  
  
getData();
```

Async/Await



Nesse exemplo, a função `getData` é definida como `async`. Ela faz uma solicitação HTTP usando `await fetch` para obter dados de uma API externa. Em seguida, `response.json()` é chamado para obter o corpo da resposta como um objeto JavaScript.

Se a solicitação for bem-sucedida (`response.ok` é `true`), os dados são impressos no console. Caso contrário, um erro é lançado. Dentro da função `getData`, usamos `try/catch` para capturar erros. Se ocorrer algum erro durante o processo de solicitação ou obtenção dos dados, ele será capturado no bloco `catch` e exibido no console.

Em resumo, o assincronismo em JavaScript é uma técnica poderosa para lidar com operações assíncronas de maneira mais legível e estruturada. Essa abordagem contribui para um código mais claro, melhor tratamento de erros e uma experiência de programação mais agradável.